Assignment 1 Performance Study

BLAS, Pthreads, CPU Affinity, Sparse Matrix Vector Multiplication

Student: Gri Raj Roll number: 23I-0601 Section: A Fall 2026

Scope and evidence. This report documents the complete implementation and the exact measurement protocol. The included CSV files are intentionally headers until they are generated on the target Linux machine: this avoids presenting fabricated performance or counter evidence. Run the supplied script, then replace the machine and measured-result placeholders before submission.

Machine and method

Item	Record from target machine
CPU topology	Paste lscpu output: model, sockets, cores, threads, cache sizes, NUMA nodes.
OS, compiler, library	Linux distribution; gcc version; OpenBLAS version; kernel version.
Timer and trials	clock_gettime(CLOCK_MONOTONIC); five trials per task/version/thread count.
Thread counts	1, 2, 4, 8, ... through available logical CPUs.
OpenBLAS control	OPENBLAS_NUM_THREADS=1 and OPENBLAS_DYNAMIC=0 for the reference comparison.

Implementation summary and correctness

Task	V1	V2	V3 and reference
1 DAXPY	Serial y = alpha*x + y.	Contiguous vector blocks.	Same static blocks, pthread affinity; cblas_daxpy reference.
2 DGEMV	Row-major output-row loop.	Disjoint contiguous output rows.	Pinned static row blocks; CblasRowMajor cblas_dgemv.
3 DGEMM	i-j-k baseline and i-k-j locality variant.	Disjoint C row blocks.	Pinned static row blocks; CblasRowMajor cblas_dgemm.
4 CSR SpMV	CSR row loop.	Disjoint contiguous CSR row blocks.	Pinned static row blocks; serial CSR reference.
5 Integration	CSV aggregation.	Average/minimum and speedup.	Efficiency and cross-kernel interpretation.

Dense data use a fixed seed (42). Tasks 1-3 compare each result with its CBLAS reference using maximum relative error <= 1e-12; this accommodates harmless floating-point summation-order differences. Task 4 uses an independently calculated serial CSR reference. Threads only write private output ranges; pthread_join is the sole completion synchronization. V3 maps worker t to logical CPU t modulo online CPUs and records the static blocked strategy.

Decomposition, locality, and arithmetic intensity

DAXPY performs about two floating-point operations while streaming x and y, so its arithmetic intensity is roughly 2/24 = 0.083 FLOP/byte absent cache reuse; it is bandwidth-bound and thread speedup will saturate near memory bandwidth. DGEMV performs about 2mn FLOPs but streams the mn matrix entries, approximately 2/8 = 0.25 FLOP/byte before vector/cache effects. Its output-row decomposition avoids reductions and gives contiguous row accesses.

DGEMM has approximately 2mnk FLOPs. For square n and ideal reuse, its arithmetic intensity grows approximately as n/12 FLOP/byte, making it far more compute/cache-reuse friendly. In row-major storage, i-k-j holds A[i,k] in a scalar and walks B[k,*] and C[i,*] contiguously; i-j-k walks B by columns and typically has poorer locality. OpenBLAS can additionally use packed panels, cache blocking, SIMD microkernels, and tuned prefetching.

CSR SpMV is dominated by irregular value/column-index streams and gathers from x. Its useful work is about 2*nnz FLOPs, while values, indices, and x accesses cause low intensity and cache/TLB pressure. Static row blocks are simple and preserve CSR row locality, but unequal nnz per row may load-imbalance the workers. A

nnz-balanced row partition or dynamic chunks is a justified follow-up if measured imbalance is large.

Performance evidence and calculation

Populate the supplied raw CSV files with the script. For each task/version/thread count, report the five raw times, average, and minimum. Use the V1 serial average as T1. Speedup S(p)=T1/Tp and efficiency E(p)=S(p)/p. Use the minimum only as a noise-resistant best-case companion; do not substitute it for the speedup baseline unless stated consistently.

Required evidence	Insert after running on target Linux
Timing results	Five trials per configuration from results/task1.csv through results/task4.csv. results/task5.csv is generated summary.
Speedup and efficiency plots	Plot S(p) and E(p) from task5.csv; identify saturation/decline.
perf stat	For serial and selected V3: task-clock, cycles, instructions, cache-references, cache-misses, context-switches, cpu-migrations. Mark unsupported events NA with perf error.
gprof optional evidence	Build with -pg in a separate profile build; report dominant function and percentage.
Affinity evidence	List requested CPU mapping and note any set-affinity error/CPU-set restriction.

Interpretation guide

A rising cache-miss rate, low instructions per cycle, and early speedup saturation support a memory-bound interpretation for DAXPY, DGEMV, and SpMV. DGEMM should show greater reuse and normally retains useful scalability longer; declining efficiency at high thread counts can arise from bandwidth saturation, cache capacity/conflict misses, OS interference, thread creation overhead, or NUMA placement. CPU affinity may reduce migrations and run-to-run variance, but it does not change the algorithm and may harm results if it crosses NUMA domains or competes with other load.

If V3 is faster or less variable than V2 while cpu-migrations fall, placement is a plausible contributor. If it does not improve, report that honestly and check cpusets, NUMA, and thermal frequency. A lower average time from OpenBLAS alone does not prove better parallel scheduling: in this experiment it also reflects packing, blocking, SIMD, and hand-tuned kernels. Keep single-thread OpenBLAS separate from any optional OpenBLAS multithread run.

Conclusion to finalize after measurement

Replace this paragraph with conclusions that cite the measured CSV and perf values. A defensible conclusion ranks kernels by observed arithmetic intensity/locality: DAXPY and SpMV generally saturate memory bandwidth first, DGEMV remains matrix-streaming limited, and DGEMM benefits most from cache reuse and optimized BLAS. State any exception shown by this machine rather than forcing the expected narrative.